

Brute Force for the 0/1 Knapsack Problem: Search Trees, Exponential Cost, and `maxVal`

This lecture converts the knapsack choice — take or leave each item — into a binary decision tree whose leaves enumerate every subset, measures the exponential cost of exploring it depth-first, and implements the brute-force solver as a recursive function `maxVal`.

The 0/1 knapsack problem

The knapsack problem is a problem in combinatorial optimization: someone constrained by a fixed-size knapsack must fill it with the most valuable items. It arises naturally in resource allocation whenever a decision-maker must choose among non-divisible projects or tasks under a fixed budget or time constraint — you cannot take half a project, just as our backpacker cannot take half a slice of pizza. The problem has been studied for more than a century, with early work dating back to 1897.

The version we are solving is the **0-1 knapsack problem**, which restricts the number of copies x_i of each item to zero or one. Formally, given n items with weights w_i and values v_i , and a maximum capacity W , the goal is to maximize the total value subject to the weight constraint:

$$\max \sum_i v_i x_i \quad \text{subject to} \quad \sum_i w_i x_i \leq W, \quad x_i \in \{0, 1\}$$

In our running example the “weight” is calories: each menu item (beer, pizza slice, burger) has a value and a calorie count, and the backpack has a calorie limit. Two related variants exist: the **bounded** knapsack problem allows up to c copies of each item, and the **unbounded** version places no upper bound on copies at all — neither applies here, since each food item is a single indivisible object.

A closely related special case is the **subset sum problem**, where each item’s weight equals its value ($w_i = v_i$); in cryptography, “knapsack problem” often refers specifically to this variant, which is one of Karp’s 21 NP-complete problems.

The problem matters widely in practice: cutting raw materials with minimal waste, selecting investments and portfolios, selecting assets for asset-backed securitization, generating keys for the Merkle–Hellman and other knapsack cryptosystems, and even constructing exams — Feuerman and Weiss proposed giving students a heterogeneous test with 125 possible points and using a knapsack algorithm to pick, from the subsets totaling exactly 100 points, the one yielding each student the highest score. A 1999 study of the Stony Brook University Algorithm Repository ranked the knapsack problem 19th most popular out of 75 combinatorial algorithm problems, and third most needed after suffix trees and bin packing.



Source: Wikipedia, article "[Knapsack problem](#)".

The search tree: enumerating every possibility

To turn the problem into a search, build a tree where **each level corresponds to making a decision about one item**. At the root, the backpack is empty and all three items remain on the table. For the first item there are exactly two choices — take it or don't — so the root has two children. Each child then branches again for the next item, and so on down the levels.

The crucial structural fact: by the time we reach the leaves, **every possible combination of items has been enumerated exactly once**. With n items there are 2^n subsets, and the tree has exactly 2^n leaves, one per subset. In the lecture's three-item example, each leaf is annotated with the total value and total calories of its combination, as read off in lecture:

Combination	Value	Calories
Beer + pizza + burger	170	766
Beer + pizza	120	766
Beer + burger	140	508
Beer only	90	145
Pizza + burger	80	612
Pizza only	30	258
Burger only	50	354
Nothing	0	0

(A side note on drawing conventions: computer scientists draw trees upside down relative to botany — root

at the top, leaves at the bottom — so “the leaves are at the bottom of the search tree” refers to this inverted picture.)

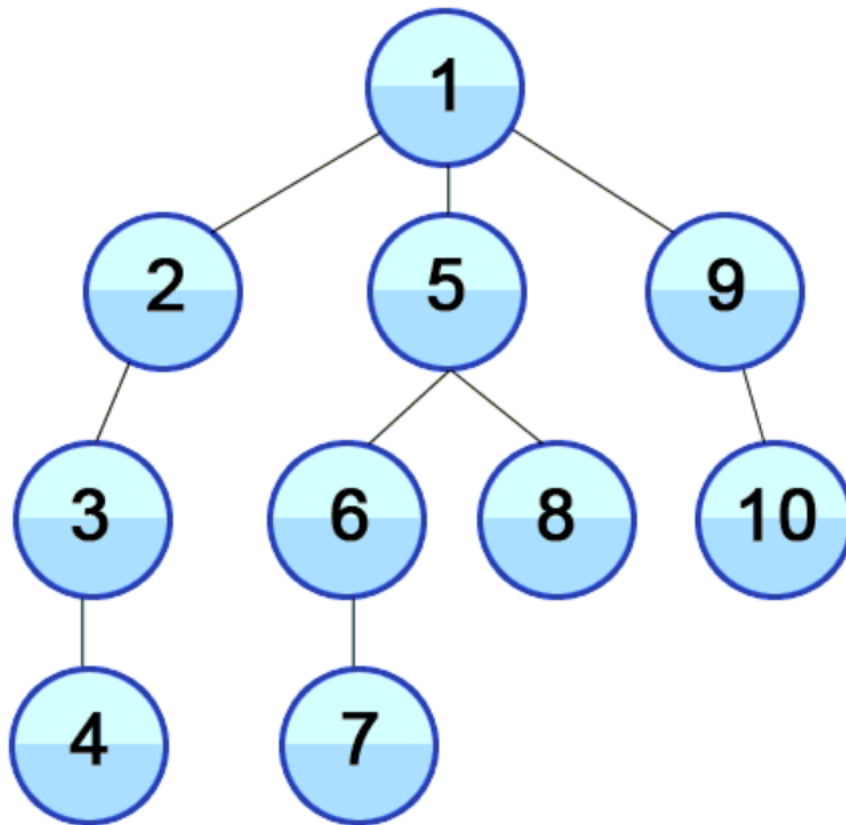
Exploring the tree depth-first

Having built the tree, we must decide an exploration order. The lecture chooses a **left-first, depth-first enumeration**: follow the leftmost path all the way down (take the beer, take the pizza, take the burger), evaluate that leaf, then back up and try the next alternative, systematically working through the entire tree.

This is precisely **depth-first search (DFS)**: an algorithm that starts at the root and explores as far as possible along each branch before backtracking, using extra memory — usually a stack — to keep track of the nodes discovered along the current branch so it can backtrack. The idea goes back to the 19th century, when Charles Pierre Trémaux used a version of it as a strategy for solving mazes.

Two properties of DFS matter for us:

- **Time**: traversing an entire structure takes time linear in its size, $O(|V| + |E|)$ for a graph — so here, time is proportional to the number of tree nodes generated, which we count below.
- **Space**: DFS needs space for the stack of nodes on the current search path. When the structure is too large to visit entirely, search is performed to a limited depth, and the space cost is only proportional to the *depth limit* — vastly less than breadth-first search at the same depth. Our recursion exploits exactly this: the Python call stack plays the role of the DFS stack, holding one frame per level, so the implicit search uses only $O(n)$ space despite visiting exponentially many nodes. DFS also lends itself well to heuristics for choosing a likely-looking branch, and when an appropriate depth limit is unknown, iterative deepening applies DFS repeatedly with increasing limits at only a constant-factor time overhead (because nodes per level grow geometrically).



Source: Wikipedia, article “[Depth-first search](#)”.

Computational complexity: exponential, and pruning doesn’t fix it

We measure time by the number of nodes generated. The tree has one level per item, and every node branches into two (take / don’t take), so the number of nodes at level i is 2^i . Summing over all levels:

$$\sum_{i=0}^n 2^i = 2^{n+1} - 1 = O(2^{n+1})$$

Exponential — which is unsurprising once you see that the tree enumerates every subset of the items, and there are 2^n subsets.

There is an obvious optimization: **prune branches that violate the constraint**. If the items already packed exceed the calorie limit, everything below that node is also infeasible, so there is no point continuing down that branch. This is a real saving in practice — but it does **not** change the complexity class: in the worst case the search is still exponential.

This exponential behavior is not an artifact of naive coding; it reflects genuine hardness. The knapsack problem is **weakly NP-complete** when weights and profits are integers, and **strongly NP-complete** when they are rational numbers — though the rational case still admits a fully polynomial-time approximation scheme (FPTAS). Hardness also depends on the computational model: NP-hardness relates to models like

the Turing machine where the size of integers matters, whereas decision trees count each decision as a single step — and even there, Dobkin and Lipton proved a $\frac{1}{2}n^2$ lower bound on *linear* decision trees (those testing the sign of affine functions) for knapsack, later generalized to algebraic decision trees by Steele and Yao. Research literature accordingly studies what makes instances “hard,” partly to build public-key systems like Merkle–Hellman on that hardness. Finally, the decision and optimization versions are polynomially equivalent: a polynomial algorithm for one yields one for the other, so both carry the same difficulty.

So does exponential complexity mean brute force is never useful? Not necessarily — it is often worth implementing the simple thing first and measuring how it behaves before investing in something cleverer.

Implementing brute force: `maxVal`

The brute-force solver is a single recursive function:

```
def maxVal(toConsider, avail):
    """toConsider: items not yet considered
       avail:       space still available
       returns:     (total value, tuple of items chosen)"""
    if toConsider == () or avail == 0:
        result = (0, ())
    elif toConsider[0].getUnits() > avail:
        result = maxVal(toConsider[1:], avail)
    else:
        nextItem = toConsider[0]
        withVal, withToTake = maxVal(toConsider[1:], avail - nextItem.getUnits())
        withVal += nextItem.getValue()
        withoutVal, withoutToTake = maxVal(toConsider[1:], avail)
        if withVal > withoutVal:
            result = (withVal, withToTake + (nextItem,))
        else:
            result = (withoutVal, withoutToTake)
    return result
```

The two parameters carry precise meanings that the whole recursion hangs on:

- `toConsider` is the list of items that nodes higher up in the tree — corresponding to earlier calls in the recursive call stack — have **not yet considered**;
- `avail` is the amount of space (here, calories) still available in the knapsack.

The body mirrors the tree’s three kinds of nodes:

1. **Base case:** if `toConsider` is empty (nothing left to consider) or `avail` is 0 (no room left), return `(0, ())` — no value, no items.
2. **Forced skip:** if the first item’s units exceed what’s available, it cannot be taken; the answer is simply `maxVal` on the rest of the items with unchanged availability.
3. **Real decision:** otherwise explore both branches exactly as the tree does. The “take” branch recurses on the remaining items with `avail` reduced by the item’s units and adds the item’s value (`withVal, withToTake`); the “don’t take” branch recurses with availability unchanged (`withoutVal, withoutToTake`). Whichever value is larger determines the returned solution, prepending the item if it was taken.

Two observations complete the picture. First, **this code never builds the search tree as a data structure** — the recursion implicitly explores it, with each call site corresponding to a node and each recursive call to a branch. Second, the local variable `result` records the best solution found so far at each point in the recursion, and it is this value that propagates up the call chain, so the root call returns the optimal solution over all 2^n leaves. Whether that exponential cost is fatal in practice is exactly what the implementation lets us test empirically.

Overlapping Subproblems: The Past Is Irrelevant

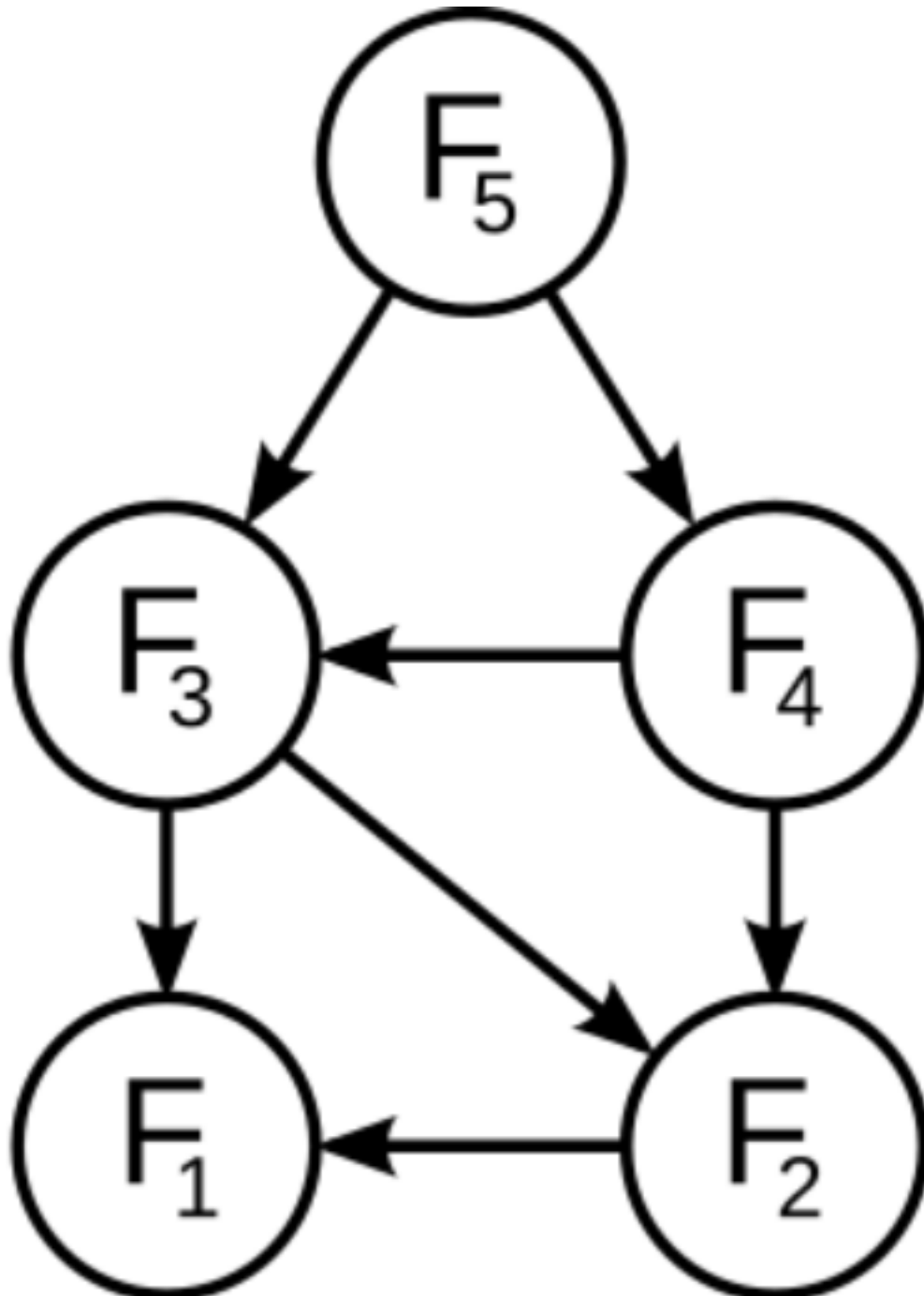
The pivotal question about the knapsack decision tree is: **what problem is being solved at each node?** The answer is remarkably spare — *given the remaining weight available, maximize the value by choosing among the remaining items*. From this follows the crucial observation: the set of previously chosen items, and even the value accumulated so far, does not matter. Any two choices made downstream of the same node inherit the same accumulated value, so that value shifts every downstream outcome equally and cannot change which future choice is best. All that matters for the future is how much weight is left and which items are still on the table.

Trace the tree for the four items a, b, c, d with capacity 5, writing each node as *(taken, remaining, value so far, weight left)*:

- **Node 0** (root): $(\emptyset, [a, b, c, d], 0, 5)$ — nothing taken, everything available.
- Take $a \rightarrow$ **node 1**: $(\{a\}, [b, c, d], 6, 2)$.
- Skip $a \rightarrow$ **node 6**: $(\emptyset, [b, c, d], 0, 5)$.

Branching continues this way down to leaves where no items remain. Now compare the two red-boxed nodes: **node 2** is $(\{a\}, [c, d], 6, 2)$ and **node 7** is $(\{b\}, [c, d], 7, 2)$. Different histories (one took a , the other took b), different accumulated values (6 versus 7) — but identical futures: the same items left to consider and the same available weight. The subtree hanging below node 2 is *exactly* the subtree below node 7. Computing it twice is pure waste. These are **overlapping subproblems**, and they are the opening dynamic programming waits for.

This is precisely the phenomenon Richard Bellman’s method exploits. Dynamic programming, developed in the 1950s and applied everywhere from aerospace engineering to economics, is both a mathematical optimization method and an algorithmic paradigm: simplify a complicated problem by breaking it into simpler sub-problems recursively. A problem has **optimal substructure** if it can be solved optimally by breaking it into sub-problems and recursively finding optimal solutions to those sub-problems; when sub-problems nest recursively inside larger problems, the relation between the larger problem’s value and its sub-problem values is called the **Bellman equation**. The classic picture of overlap is the Fibonacci computation — its subproblem graph is *not a tree*, because many call paths converge on the same subproblems:



Source: Wikipedia, article [“Dynamic programming”](#).

The knapsack tree exhibits the same shape: exponentially many *paths*, but far fewer distinct *problems* at the nodes.

Memoization: The fastMaxVal Recipe

The fix is to stop recomputing shared subtrees. Modify `maxVal` to carry a **memo** — this is dynamic programming in its top-down form:

```
def fastMaxVal(toConsider, avail, memo={}):
```

Three design decisions make it work:

1. **The key.** The memo is keyed by the pair (*items left to consider, available weight*). Because decisions proceed front-to-back and never revisit earlier items, the items left are always a **suffix** of the original list — so the suffix can be represented compactly by `len(toConsider)`.
2. **Check first.** The very first thing the function body does is look up whether the optimal choice for this available weight is already in the memo; if so, return it and do no work at all.
3. **Update last.** The last thing the body does is store the freshly computed answer in the memo.

Together these enforce the invariant that each distinct problem is solved exactly once. Conceptually this matches the canonical dynamic-programming formalism: define value functions $V_1, V_2, \dots, V_n$ over the state of the system, relate each to the next through the Bellman-equation recursion, and recover the optimal decisions by tracking back through the recorded calculations. The memo simply performs that bookkeeping on demand, caching each state’s value the first time it is needed.

What Memoization Buys: Exponential Leaves, Linear Calls

The payoff is dramatic. Comparing the number of leaves exhaustive enumeration would visit (2^n) against the calls the memoized version actually makes:

Items	Exhaustive leaves (2^n)	Memoized calls
2	4	7
4	16	25
8	256	427
16	65,536	5,191
32	4,294,967,296	22,701
64	18,446,744,073,709,551,616	42,569
128	“Big”	83,319
256	“Really Big”	176,614
512	“Ridiculously big”	351,230
1024	“Absolutely huge”	703,802

Two things stand out. First, the crossover: around 8–16 items the memoized version overtakes exhaustive search, and by 32 items exhaustive enumeration faces over four billion leaves while memoization makes about twenty-three thousand calls. Second, the growth pattern: **double the items, roughly double the calls** — the call count grows essentially linearly, not exponentially.

Why It Works — and Why It Isn’t a Miracle

If the knapsack problem is exponential, have we overturned the laws of the universe? No — computational complexity is subtler than the size of a decision tree. The running time of `fastMaxVal` is governed by the number of **distinct pairs** (`toConsider, avail`):

- Possible values of `toConsider` are bounded by `len(items)` — easy.
- Possible values of `avail` are bounded by the **number of distinct sums of weights** — which is why the memo never needs more than that many entries.

We are not exploring an exponential number of *different* problems, only an exponential number of *paths* leading to a much smaller set of problems — and the memo guarantees each is solved once. This also explains why the trick depends on the input’s form. The 0-1 knapsack problem asks, for items with weights w_i and values v_i and capacity W :

$$\text{maximize } \sum_{i=1}^n v_i x_i \quad \text{subject to } \sum_{i=1}^n w_i x_i \leq W, \quad x_i \in \{0, 1\}.$$

Its hardness depends on how weights and profits are given: with **integer** weights and profits it is *weakly* NP-complete, while with **rational** ones it is *strongly* NP-complete (though even then it admits a fully polynomial-time approximation scheme). The distinct-sums bound is the integer-case lever: the algorithm's cost scales with the *magnitude* of the weights — how many reachable capacity values exist — rather than with the item count alone, which is exactly the behavior weakly NP-hard problems permit. Related hardness facts round out the picture: the decision and optimization versions are polynomially inter-reducible (a polynomial decision algorithm yields the optimum by iterating on the threshold k , and vice versa); the special case where $w_i = v_i$ is the subset sum problem, one of Karp's 21 NP-complete problems and the basis of knapsack cryptosystems such as Merkle–Hellman; and even in decision-tree models that count each test as one step, Dobkin and Lipton proved a $\frac{1}{2}n^2$ lower bound for linear decision trees, later generalized to algebraic decision trees by Steele and Yao. None of this is surprising given the problem's reach: studied for over a century (early work dates to 1897), it models cutting raw material with minimal waste, investment and portfolio selection, asset-backed securitization, exam design (Feurman and Weiss used it to pick, from a 125-point heterogeneous test, the 100-point subset giving each student the highest score), and it ranked 19th most popular — and third most needed — among 75 problems in a 1999 survey of the Stony Brook Algorithm Repository.

The Big Picture: Four Takeaways on Optimization Algorithms

Pulling both lectures together:

1. **Many practical problems are optimization problems.** A huge fraction of real-world questions recast as: here is an objective function, here are constraints, find the best solution.
2. **Greedy algorithms are often adequate — but not necessarily optimal.** A greedy algorithm makes the locally optimal choice at each step and never reconsiders past choices. Uriel Feige calls it “the ultimate form of dynamic programming, in which only one partial solution is maintained” — a special case of DP that demands far more structure from the problem. Sometimes it is exact, as in activity selection, solvable in $O(n \log n)$ by sorting tasks by end time and repeatedly taking the first task that begins after the last one ends; the classic algorithms built on greedy properties include Huffman coding, Prim's, Kruskal's, and Dijkstra's. Theory pinpoints when greedy works — Jack Edmonds identified matroid-structured problems, and Korte and Lovász broadened this with greedoids (which yield, for example, a proof of Prim's optimality) — and optimality proofs typically use an **exchange argument**: show any solution differing from the greedy one is at most as good. Note that algorithms undoing past steps don't qualify (Gale–Shapley modifies existing pairings, so it is not greedy). More often greedy serves as an *approximation*: greedy vertex coloring gives the bound $\chi(G) \leq \Delta(G) + 1$; a cheapest-edge greedy for traveling salesman stays within $\Theta(\log n)$ of optimal; applying the fractional-knapsack greedy to 0-1 knapsack guarantees at least half the optimal value, as does greedy submodular maximization; and the technique extends to set cover, load balancing, Steiner tree, and independent set. Fast and easy to implement, yes — but it can leave you standing on a platform that isn't the top of the hill.
3. **Finding an optimal solution is usually exponentially hard.** If the only way to be certain is to enumerate all possibilities, running time blows up as the problem grows.
4. **Dynamic programming delivers good performance for the subclass with optimal substructure and overlapping subproblems** — and it comes with two key properties: the answer is *always correct* (the true optimum, not an approximation), and it is *fast under the right circumstances* because it never solves the same subproblem twice. Optimal substructure means the pieces genuinely compose: the textbook illustration is shortest paths, where any subpath of a shortest path is itself a shortest path, so the bold start-to-goal route decomposes into independently optimal segments:

A Parting Exercise: The Roll-Over Problem

As a closing challenge that exercises every tool from these lectures, consider the **roll-over** optimization problem. You have 60 units of effort to spend over the term. The variables a, b, c, d, e are the effort devoted to each of five problem sets, whose given weights are $ps_1, \dots, ps_5$; whatever effort you *don't* spend rolls over to the final exam, where it is multiplied by F , the value of the final:

$$\text{Score} = (60 - (a + b + c + d + e)) F + a \cdot ps_1 + b \cdot ps_2 + c \cdot ps_3 + d \cdot ps_4 + e \cdot ps_5$$

Objective: given values for $F, ps_1, ps_2, ps_3, ps_4, ps_5$, choose a, b, c, d, e to maximize the score.

Constraints: each of a, b, c, d, e is either 10 or 0 — full effort on a problem set or none — and their sum must satisfy $a + b + c + d + e \geq 20$.

Notice the family resemblance to what came before: five binary choices, a 60-unit budget in which each selected problem set consumes 10 units, unspent budget converted at rate F , and a floor requiring at least 20 units of spending. Brute force? Greedy? Dynamic programming? Work through which framework fits — and why.